Deep Learning 2

Special blocks

Machine learning

Roman Jakubíček & Jiří Chmelík
Department of Biomedical Engineering
Brno University of Technology

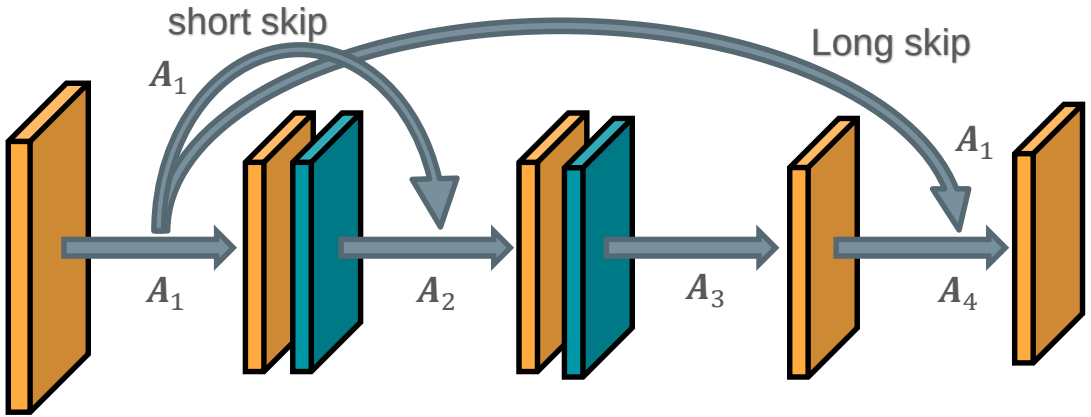
Introduction

- Other blocks and structures
  - Skip connections
  - 1×1 convolution
  - Bottleneck
  - Dilated convolution
  - Up-sampling layers
  - Residual block

Special blocks

Skip connections

- Short or long skip connections
- Skip connections help traverse information in deep neural networks
- It is trying to find and to use the layers which are rich in features
- It provides improving the information flow between layers
- Solve vanishing or exploding problems (BatchNorm helps in depth) in ”shallow” nets



Output from l^{th} layer - A_l

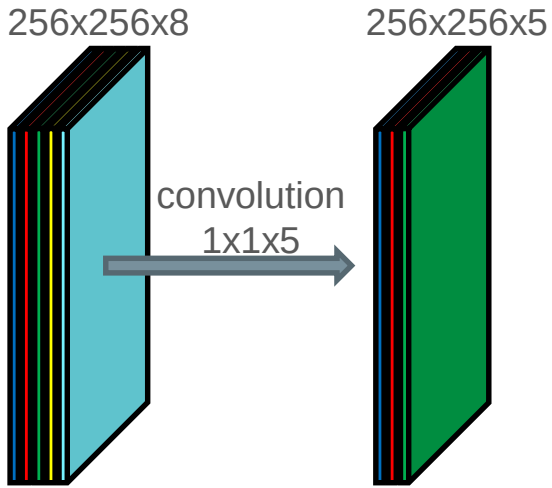
… but, how do we can join data?

1 x 1 convolution layer

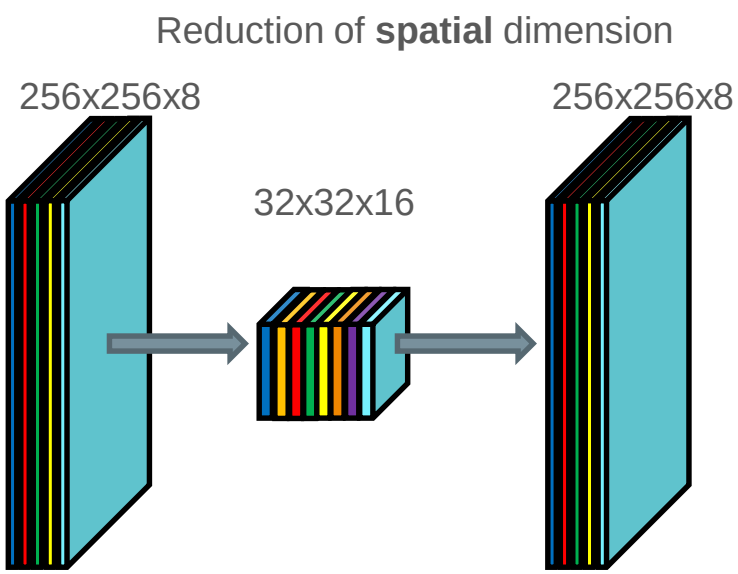
- Sometimes called “point-wise convolution”
- It is not combining spatial information, but it is combining features in depth
- The output is the same size as input in spatial domain, but the depth (number of features) can be changed.
- We obtain new activation maps (feature maps), which are a linear combination of previous features
- It has no hyper-parameters, but there are learned the weights of individual features

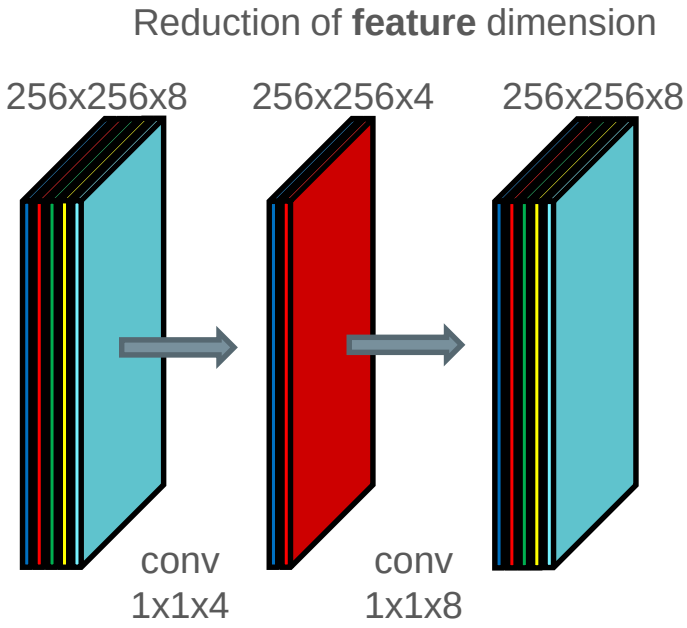
Why would we want 1x1 convolution?

- New combination features
- Reduce or expand features
- Speed up by decreasing the dimension
- Separable convolution (MobileNet)
- Alternative computation of FC layer (assuming 1x1xN tensor as input)



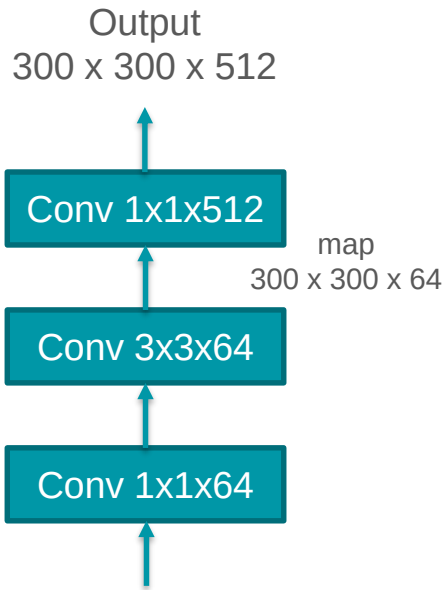
BottleNeck

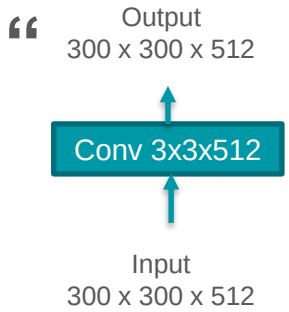

Reduction of **spatial** dimension
256x256x8
32x32x16
256x256x8


Reduction of **feature** dimension
256x256x8
256x256x4
256x256x8
conv
1x1x4
conv
1x1x8

BottleNeck layer

- for convolution neural networks
- to regularize model
- to reduce the size of the input tensor in a convolutional layer
- lead to large saving in a computational cost
- preserves information with maximal relevance
- included in GoogLeNet (Inception V2 and V3) or DenseNet





Dilated convolution

- aims to aggregating the multi-scale contextual information

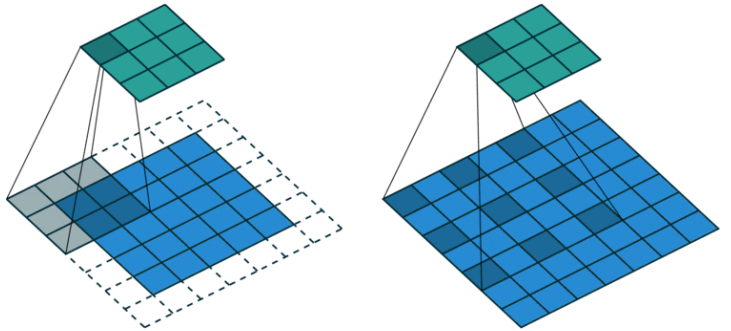
Standard convolution

$$F(\boldsymbol{p}) * k(\boldsymbol{p}) = \sum_{\boldsymbol{p}=\boldsymbol{s}+\boldsymbol{t}} F(\boldsymbol{t})k(\boldsymbol{s})$$

Dilated convolution

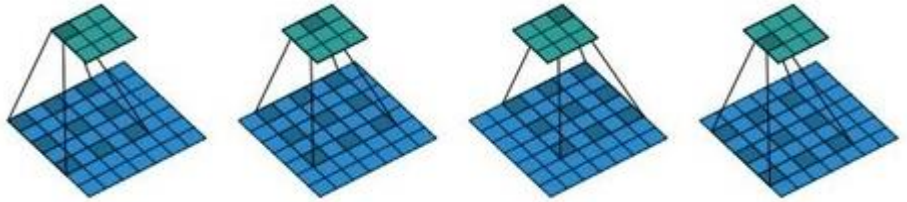
$$F(\boldsymbol{p}) *_{\delta} k(\boldsymbol{p}) = \sum_{\boldsymbol{p}=\boldsymbol{s}+\delta\boldsymbol{t}} F(\boldsymbol{t})k(\boldsymbol{s})$$

Dilation factor $\delta = 1$ $\delta = 2$



- no down-sampling
- the receptive field is larger without the need for the down and up-sampling
- analogously, the classic convolution with larger sparse kernel or pyramidal approach

Dilated convolution with δ factor = 2



Up-sampling layers

The neural networks to generate images, it usually involves up-sampling from low resolution (from depth) to high resolution (output)

Un-pooling

- suitable for resampling (up) after a max pooling layer (non-linear)

Transposed convolution

- This structure enables up-sampling matrices (features maps) from previous layers by convolution performed via matrix multiplication. Weights of convolution kernel are learned to achieved the best interpolation

Interpolation

- elementary interpolation method (bilinear)
- there is nothing that the network can learn about

Residual block

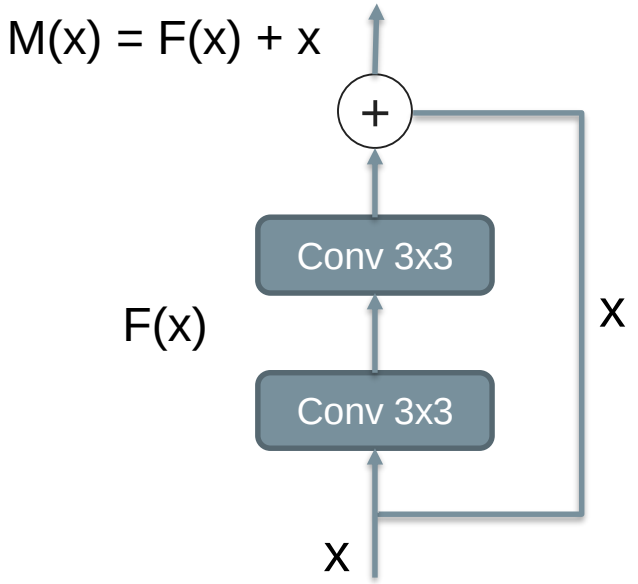
- The deeper model should be able to perform at least as well as the shallower model
- but deeper models are harder to optimize
- It has two branches flowing into additive connection (identity and convolved identity)

- Thus, the weights of conv layers are adapted this way, that the output from them will “residuum” defined as follows

$$F(x) = M(x) - x$$

- In other words, this block can take the output from a series of conv layers or just original input (skip layer)

- Avoids vanishing gradients
- May be able to learn simple function (identity function)



CNN Applications

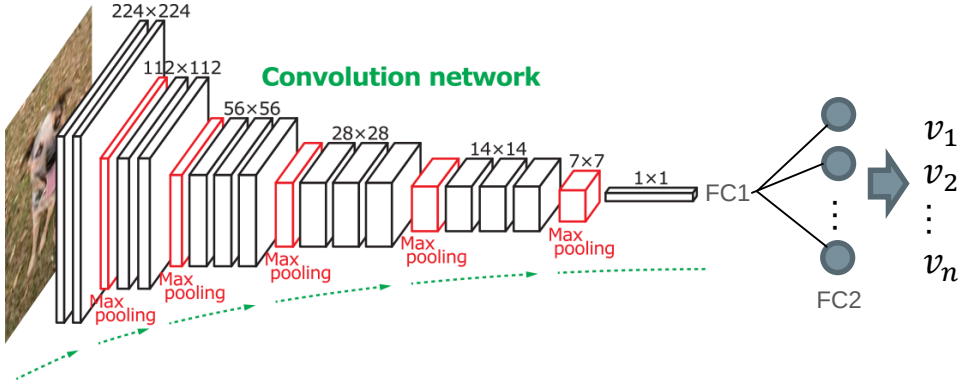
Introduction

- Application of CNN
  - Regression
  - Classification
  - Image-to-image regression, segmentation
  - Object detection (R-CNN, YOLO, SSD, …)
  - Signal and Language processing

Regression and classification

Architecture

Regression problem



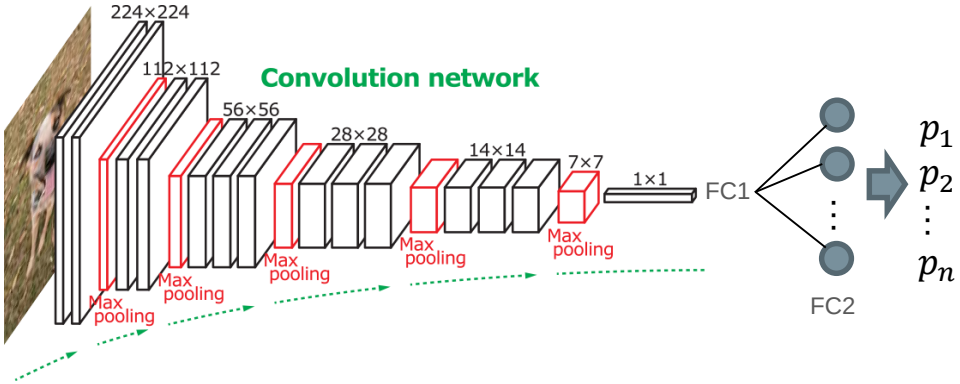
- Architecture: encoder => FC
- Input: image
- Output: regressed values

- Loss fcn: L_1 *or* L_2

- Last layer:
  - fully connected / 1x1 conv
  - no activation function (in the specific cases ReLu or softmax)
  - number of neurons corresponds to the number of regressed values

- it is sometimes appropriate that the predicted value is relative, e.g. for prediction positions,…

Classification problem



- Architecture: encoder => FC
- Input: image
- Output: vector of probabilities

- Loss fcn: Multi-class Weighted Cross-entropy

- Last layer:
  - Fully connected / 1x1 conv
  - Softmax (sigmoid)
  - Number of neurons corresponds to the number of classes

One-hot encoding
- Output vector includes values of membership probabilities for each class (length of the vector equals the number of classes)
- Resulting estimated classes is taken with maximal probabilities

Integer encoding
- Alternatively, the output can be integer number – a label of class (e.g. suitable for ordinal variable)

Encoder Architecture Comparison

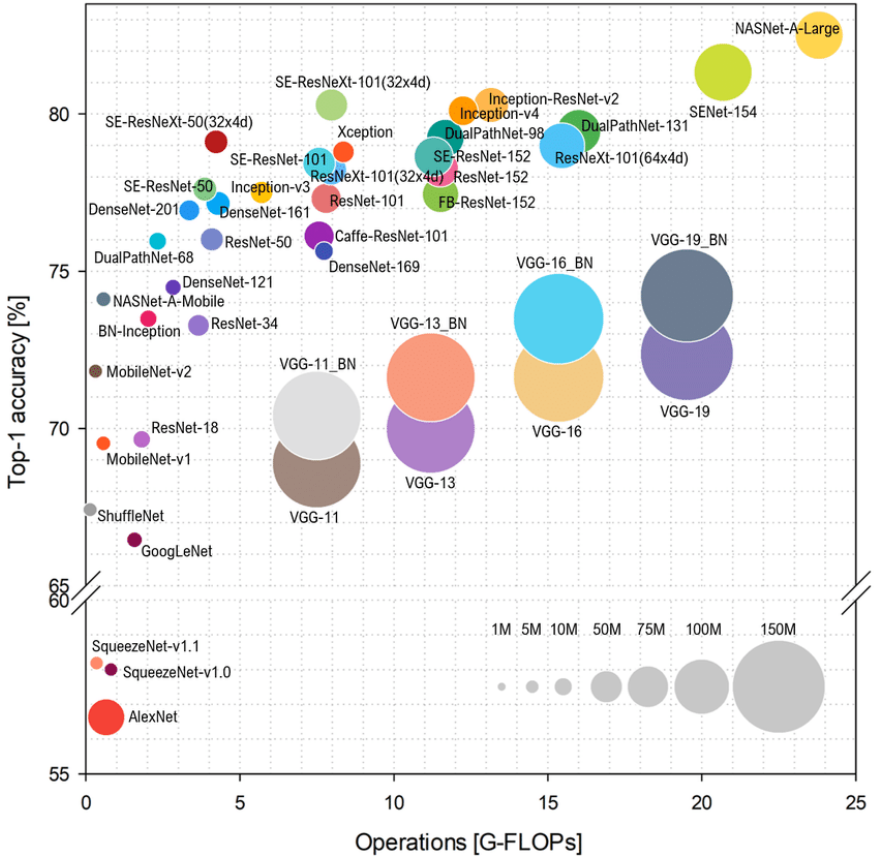
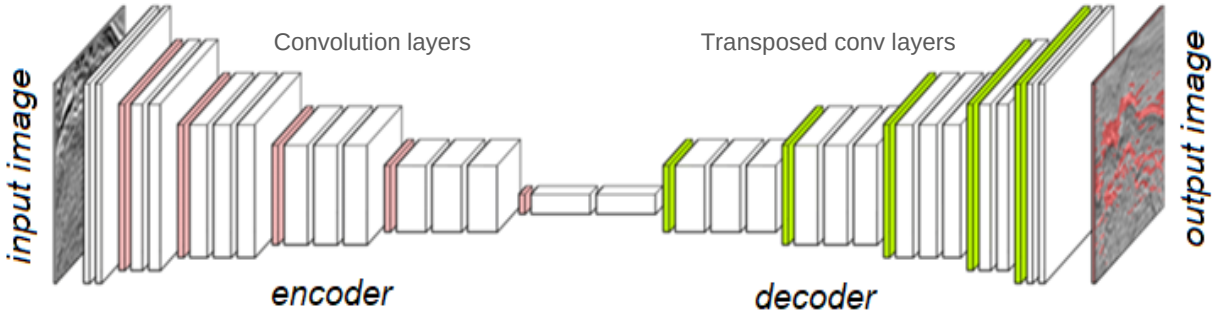


Image-to-image

Architecture

Image Regression

- Architecture: encoder – decoder
- Input: image
- Output: image
- Loss fcn: L_1 *or* L_2
- Last layer:
  - Convolution layer
  - Linear activation



- We need paired data (reference data for an input image) – can be problematic
- If we haven't these data, we can create e.g. synthetic or obtain during new acquisition
- There are existing special methods, which work with non-paired images

Image Regression

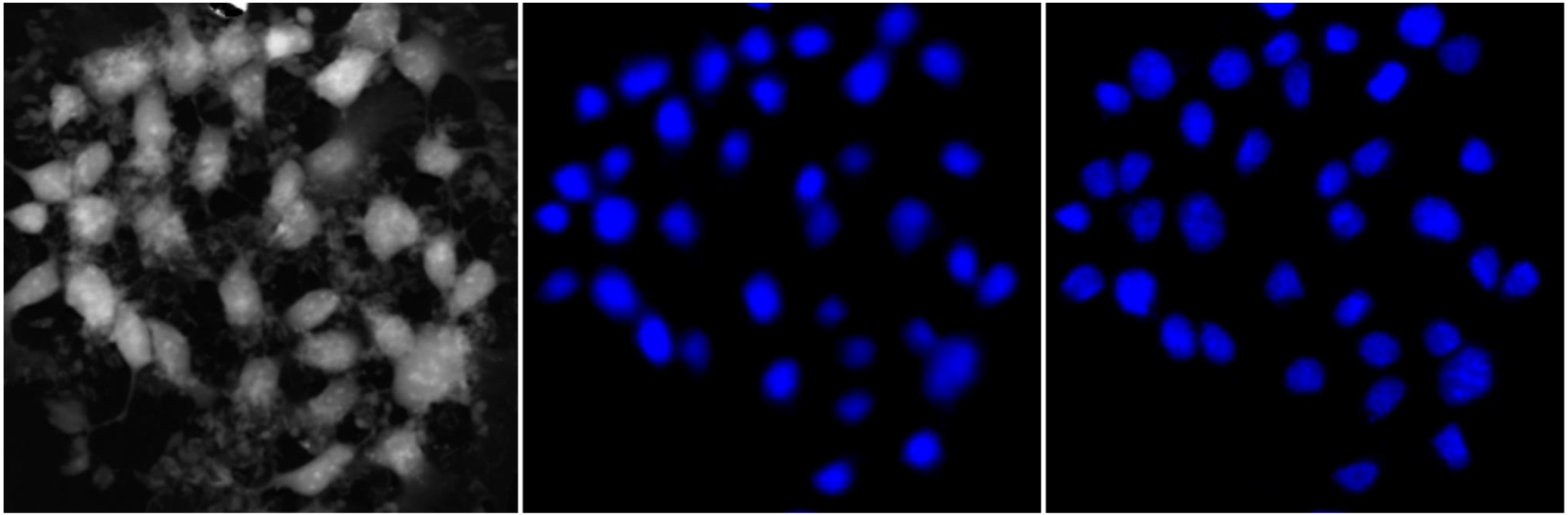
Utilization:

– image segmentation
– noise reduction
– reconstruction from sinogram
– image transformation to new image (GAN,…)

Image Regression

Example of fluorescence prediction of cells' nuclei from microscopic image

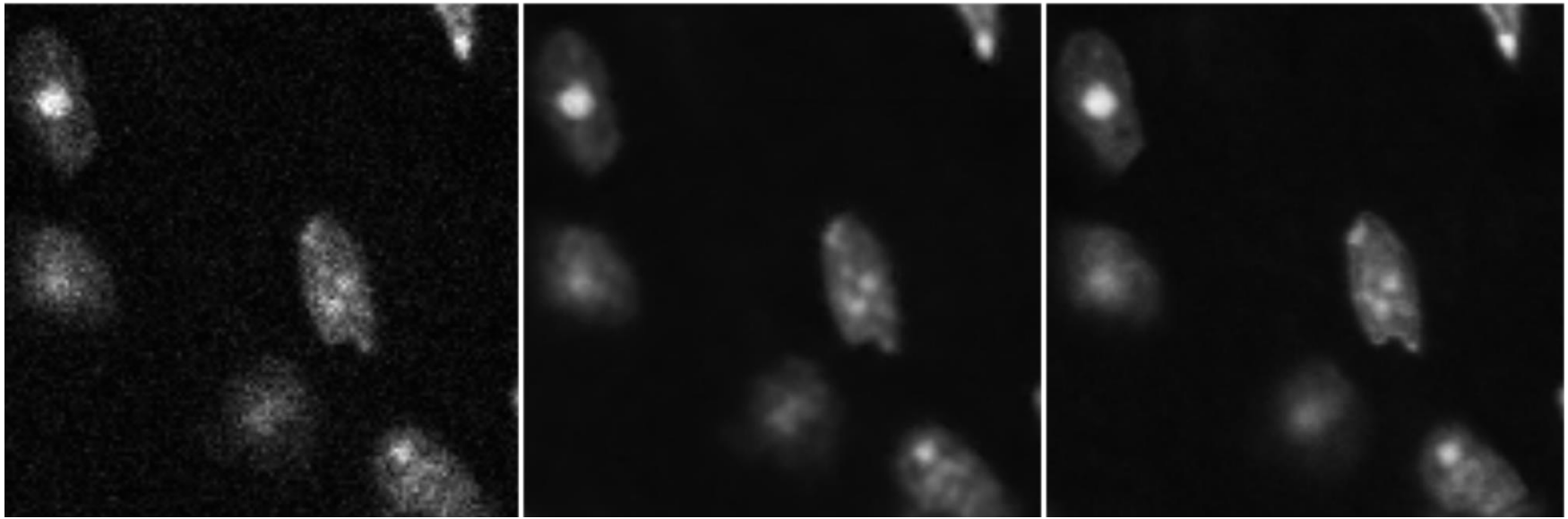
Input image | Regression image | Reference image



Noise reduction

Example of denoising

Input image | Regression image | Reference image



Next example:
https://www.youtube.com/watch?time_continue=24&v=ooeHl_qmqzI&feature=emb_logo

Noise reduction

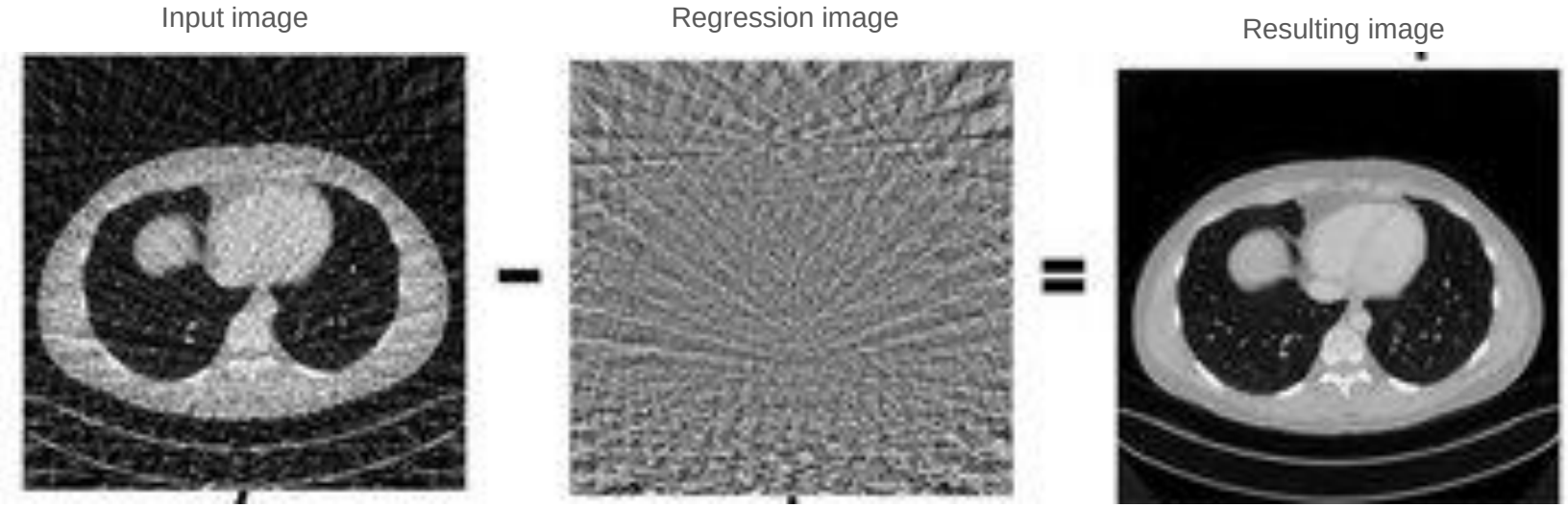


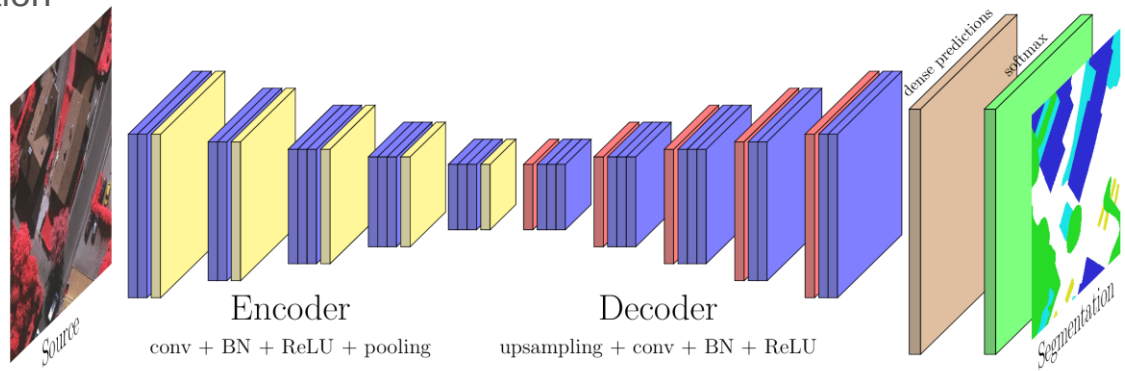
Image segmentation

A special case of the image-to-image transformation – output is a label of the classes for each pixel

- **Architecture**: encoder – decoder
- **Input**: image
- **Output**: 2D probability map for each class
- **Loss fcn**: Multi-class Weighted Cross-entropy or Binary Tverski (e.g. Dice loss)
- **Last layer:**
  - convolution layer, where its depth size = number of classes
  - softmax (through depth) for each pixel
  - alternatively 1x1 convolution

Output has size $M \times N \times D$, where D is the number of classes.

We select classes with maximal probability for each pixel.



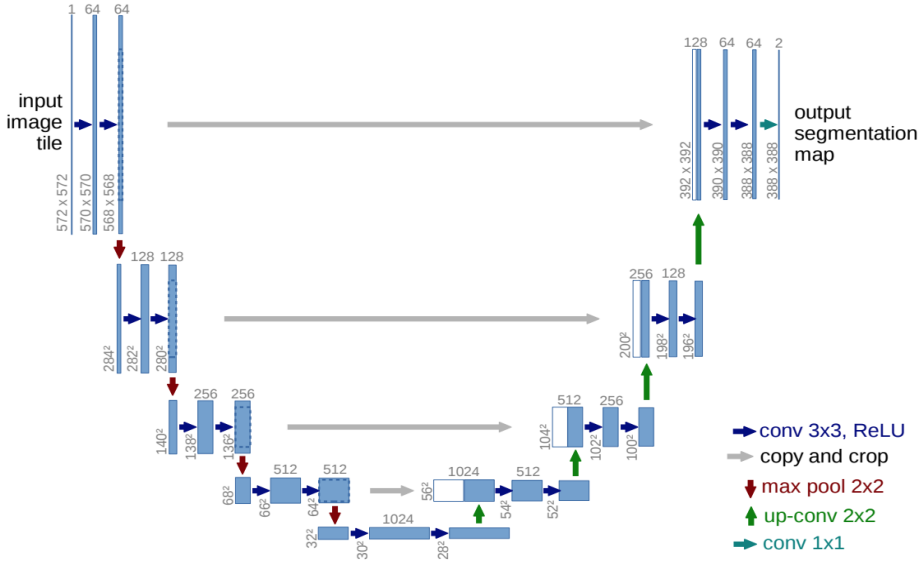
U-Net (2015)

- Developed for biomedical image segmentation
- Use long **skip connections** between the down-sampling path and the up-sampling path
- Skip connections with **concatenation**
- The bottleneck is built from two 3x1 convolutional layers (with BN and ReLu)
- It combines the location information from the down-sampling path with the contextual information in the up-sampling path

- No general pre-trained model

- There are many pre-trained models for specific segmentation problems (e.g. MRI brain)

- But, we can use e.g. VGG pre-trained model in the encoder part



Object detection

Architectures

Object detection

It is a computer vision technique that aims to detect objects in from bounding boxes. It can find objects such as cars, human, face or anatomical structure or pathologies.

Object detection usually involves two processes:

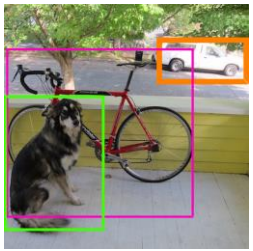
– localization of the object

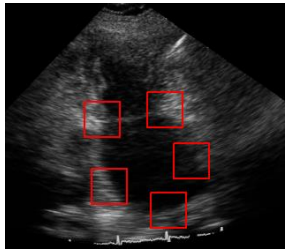
– classification of this object

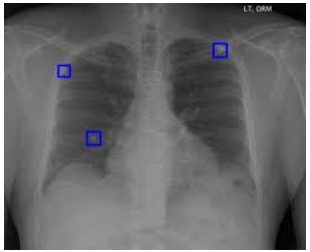
Usually, these steps can be performed iteratively, parallel or in one step.

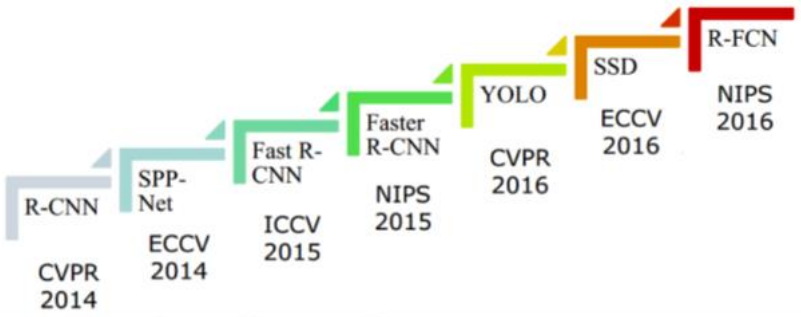
The most common algorithms:

- R-CNN (“fast”, “faster”, “cascade”)
- SSD
- YOLO (v.3)
- RetinaNet









Faster R-CNN

Region convolution neural network

- Takes an image (of any size) as input and outputs a set of rectangular object proposals, each with an objectness score
- The third version of the algorithm (R-CNN => fast R-CNN => faster R

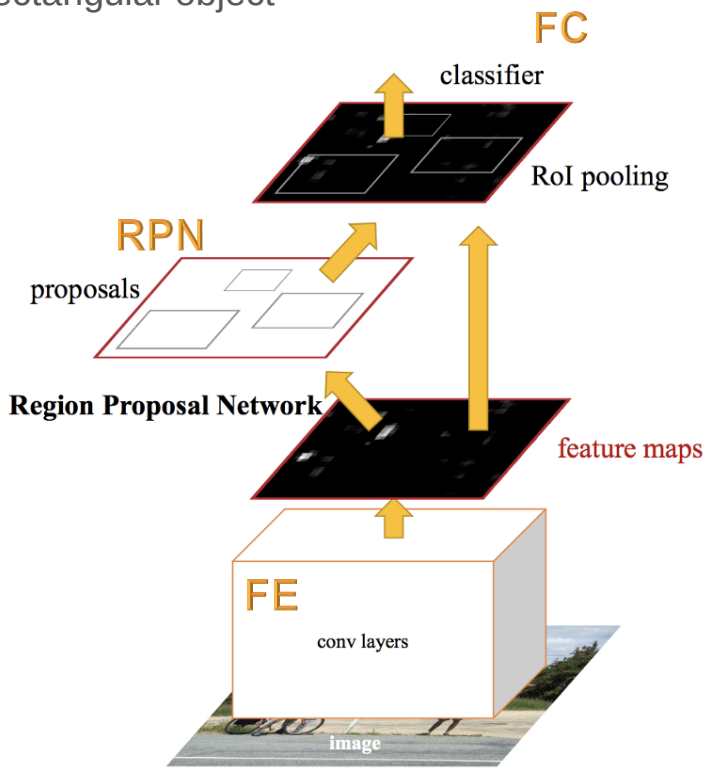
Basic two modules:

– extraction of features by CNN with FC classifier (fRCNN)

– network for proposal region (RPN)

- The iterative four-stage learning process
- Sharing convolution layers

Modules: FE – features extractor, FC – fully connected classificatory, RPN – region proposal network

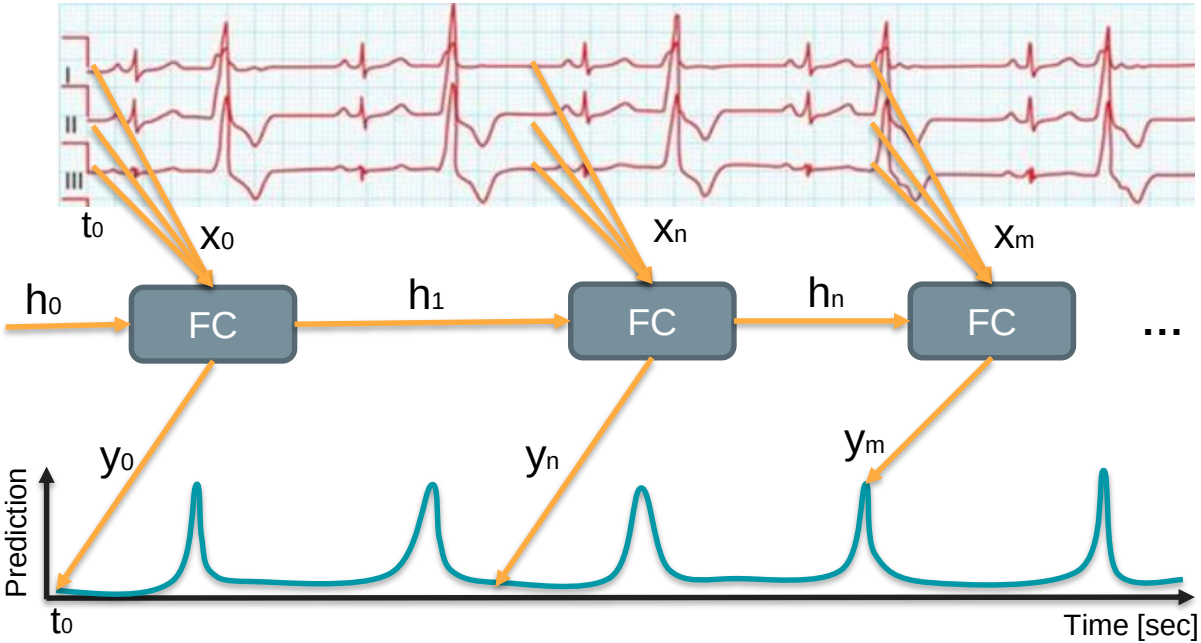


Signal and Language

Architectures

Recurrent Networks

- this network introduces new input ($\mathbf{h}$) vector, which is output from network in previous time point
- FC has thus two input vectors ($\mathbf{h}_{k-1}$ and $\mathbf{x}_{k-1}$) and two outputs ($\mathbf{h}_k$ and $\mathbf{y}_k$)
- for prediction of next time point in signals through any dimension (time), the FC net uses information in form the vector $\mathbf{h}$ from previous time point



Summary

Pros

- predictor, which uses older prediction in the form activation values (h_{i-1}) from previous “iteration”
- learned gradual forgetting older prediction
- information transfer over time (or another dimension)
- we can process sequence of different lengths
- if the input size is larger, the model size does not increase

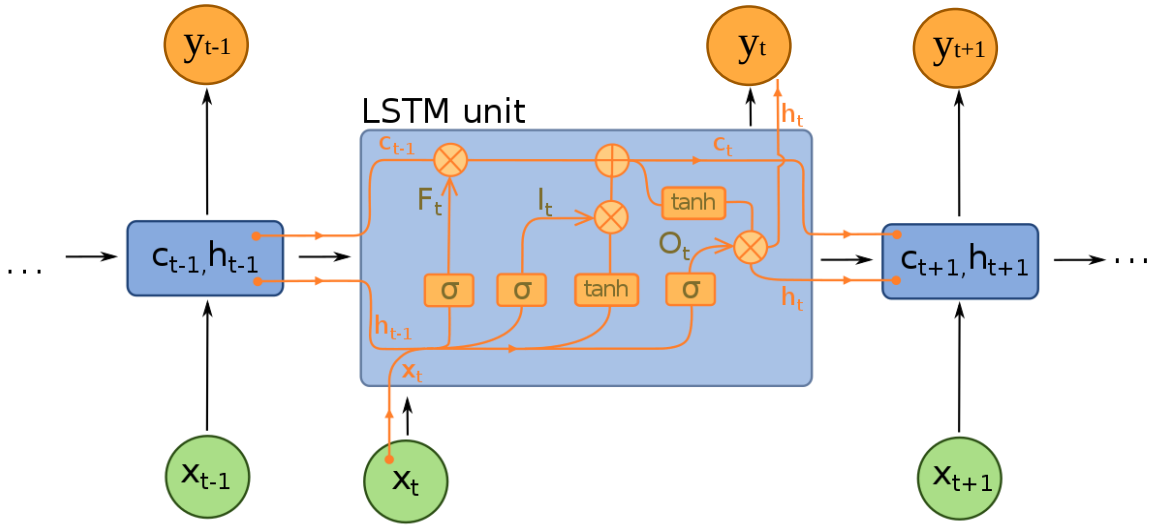
Cons

- difficult training (depend on length of signals)
- for each “time” point, the prediction of networks is required – must be performed iteratively
- gradient vanishing and exploding problems
- cannot be parallelized
- cannot be used transfer learning

There are many modifications of RNNs (e.g., network doesn’t use $\mathbf{W}_y$ or bi-directional, …)

LSTM – Long Short-Term Memory

- This network introduces into RNN the recurrent “forget gates”, which allow use very long memories.
- The LSTM approach solves vanishing gradient or exploding problem in backprop.
- It can work even given long delay between significant events



Summary

Pros

- very successful modification of RNNs and their utilizing
- solves a vanishing gradient problem (error can be backpropagated through time and layers)
- long memory
- we can process sequence of different lengths

Cons

- uses sigmoid or tanh activation function – more prone to saturation (you can use “softsign”)
- Iterative calculation – high computational complexity
- cannot be parallelized
- cannot use transfer learning
- cannot be re-trained – generalization problem

There are many modifications of the LSTM (e.g. adding “peephole connection”, Depth Gated RNNs, Clockwork RNNs, grid LSTM or **GRU**)

Utilizing

- **Natural language processing**
  - obtaining the context from sentence/text
  - classification of text document
  - automatic translation
  - spam detection

- **Speech recognition**
  - transfer spoken word to written text

- **Financial prediction, …**

- **Analysis of sequences:**
  - gene sequences (e.g. secondary structure prediction, …)
  - biomedical signals (e.g. prediction of pathological stages, …)
  - biomedical images (e.g. explain Images, video, …)

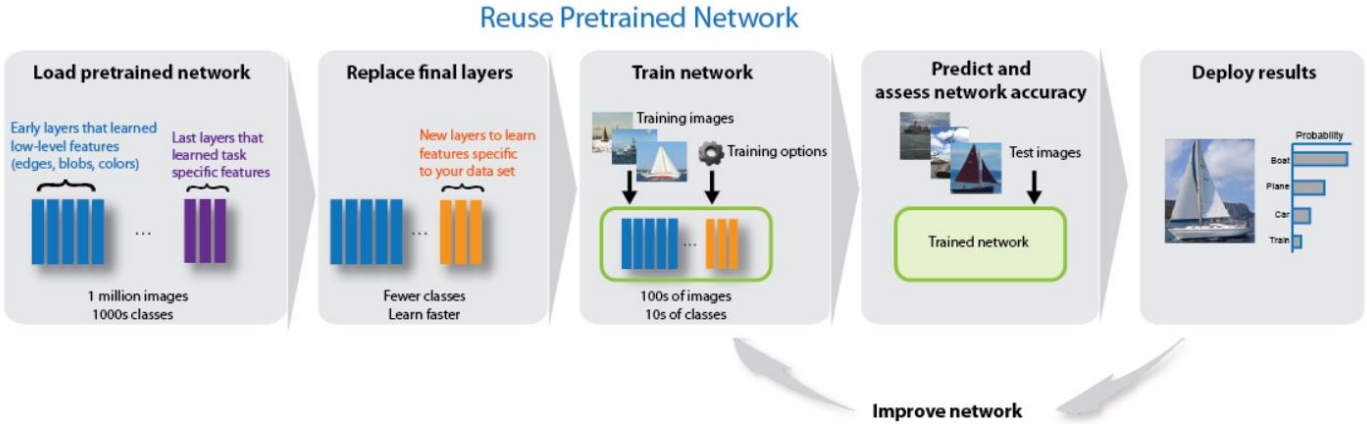
…

Tips and Tricks

Transfer learning

- It turned out, that early layers have a tendency converge to impulse responses corresponding the edge detectors (first and second derivation, edge detectors, Gabbors' or Haars' masks).
- These basic features are then propagated to following layers, and there are combining into complex (abstract) features.

- Is the process of taking a pre-trained model (the weights and parameters of a network) and fine-tuning the model with our own dataset



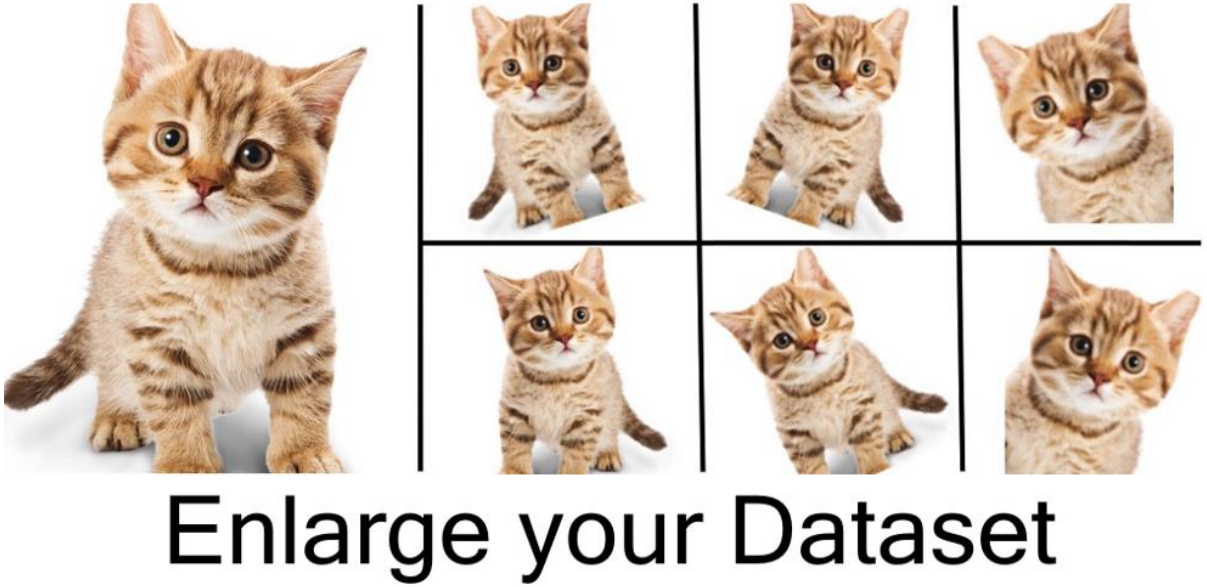
Data augmentation

- It is the process for extending the training dataset, which leads to more robust NN.
- We create synthetic variants (synthetically modified) from real images, which can occur in the real world, but it is not available in our dataset.

- **Types**:
  - geometrical transformation (flip, rotation, scaling, crop, …)
  - intensity modification (point-wise operation, color,)
  - noising
  - blurring/sharpening
  - data mixing
  - random image generation (GAN)

- **Realization**:
  - ***offline approach*** (smaller datasets) before we feed the data to the model.
  - ***online approach*** is augmentation in the fly, the transformations are performed on the mini-batches (GPU).

Data augmentation



Frameworks

Application

Research purposes – PyTorch, TensorFlow2

For beginners – Keras

For products, commercial utilizing – TensorFlow, (MXNet, Gluon)

Transfer deep learning models – ONNX

For Android developers – D4LJ

For iOS developers – Core ML